

SI1001 Teoría de la Computación

Lógica de Hoare

Andrés Sicard Ramírez

Universidad EAFIT

Semestre 2025-2

Preliminares

- El texto guía para estas diapositivas es (Grassman y Tremblay [1996] 2003, cap. 9).
- Los números asignados a las definiciones, ejemplos, ejercicios, figuras, páginas, secciones, tablas y teoremas en estas diapositivas corresponden a los números asignados en el texto guía.

Conceptos preliminares

Descripción

Emplearemos la lógica de Hoare para verificar programas de un subconjunto del lenguaje imperativo PASCAL:

- (i) operadores aritméticos,
- (ii) funciones de la biblioteca estándar,
- (iii) bloques **begin-end** ,
- (iv) instrucciones de asignación (**:=**),
- (v) instrucciones de secuenciación (**;**),
- (vi) instrucciones **if-then** y **if-then-else** ,
- (vii) instrucciones **while** .

Conceptos preliminares

Descripción

El **estado de un programa** consiste del valor de la variables utilizadas por el programa.

Conceptos preliminares

Descripción

El **estado de un programa** consiste del valor de la variables utilizadas por el programa.

Descripción

Las **sentencias (proposiciones)** de la lógica de Hoare son sentencias (proposiciones) de la lógica de predicados de primer orden en las variables del programa.

Conceptos preliminares

Notación

Para escribir las sentencias emplearemos las siguientes conectivas y constantes lógicas:

V (verdadero)

F (falso)

$\neg$ (negación)

$\wedge$ (conjunción)

$\vee$ (disyunción)

$\Rightarrow$ (condicional, implicación material)

$\Leftrightarrow$ (bicondicional, equivalencia material)

$=$ (igualdad)

Conceptos preliminares

Notación

$A \equiv> B$ (implicación lógica, es decir, $A \Rightarrow B$ es una tautología)

$A \equiv B$ (equivalencia lógica, es decir, $A \Leftrightarrow B$ es una tautología)

Conceptos preliminares

$P \vee \neg P \equiv V$	Ley de medio excluido
$P \wedge \neg P \equiv F$	Ley de contradicción
$P \wedge V \equiv P$	Leyes de identidad
$P \vee F \equiv P$	
$P \vee V \equiv V$	Leyes de dominación
$P \wedge F \equiv F$	
$P \vee P \equiv P$	Leyes de idempotencia
$P \wedge P \equiv P$	
$\neg(\neg P) \equiv P$	Ley de doble negación

Tabla 1.16: Equivalencias lógicas

(continua en la próxima diapositiva)

Conceptos preliminares

$P \vee Q \equiv Q \vee P$ $P \wedge Q \equiv Q \wedge P$	Leyes conmutativas
$(P \vee Q) \vee R \equiv P \vee (Q \vee R)$ $(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$	Leyes asociativas
$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$ $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$	Leyes distributivas
$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$	Leyes de De Morgan

Tabla 1.16: Equivalencias lógicas (continuación)

Conceptos preliminares

Notación

Un argumento válido con premisas $A_1, A_2, \dots, A_n$ y conclusión B , se denotará por:

$$A_1, A_2, \dots, A_n \models B.$$

Conceptos preliminares

$A, B \models A \wedge B$	Ley de combinación
$A \wedge B \models B$	Ley de simplificación
$A \wedge B \models A$	Ley de simplificación (variante)
$A \models A \vee B$	Ley de adición
$B \models A \vee B$	Ley de adición (variante)
$A, A \Rightarrow B \models B$	Modus ponens
$\neg B, A \Rightarrow B \models \neg A$	Modus tollens
$A \Rightarrow B, B \Rightarrow C \models A \Rightarrow C$	Silogismo hipotético
$A \vee B, \neg A \Rightarrow \models B$	Silogismo disyuntivo
$A \vee B, \neg B \Rightarrow \models A$	Silogismo disyuntivo (variante)
$A \Rightarrow B, \neg A \Rightarrow B \models B$	Ley de casos
$A \Leftrightarrow B \models A \Rightarrow B$	Eliminación de la equivalencia
$A \Leftrightarrow B \models B \Rightarrow A$	Eliminación de la equivalencia (variante)
$A \Rightarrow B, B \Rightarrow A \models B \Leftrightarrow A$	Introducción de la equivalencia
$A, \neg A \models B$	Ley de inconsistencia

Tabla 1.25: Leyes de inferencia

Conceptos preliminares

Definición 9.1

Una **aserción** (**afirmación**) es una sentencia referente a un estado de programa.

Conceptos preliminares

Definición 9.1

Una **aserción** (**afirmación**) es una sentencia referente a un estado de programa.

Notación

Las aserciones se escribirán entre llaves. Es decir, $\{A\}$ denota que A es una aserción.

Conceptos preliminares

Definición 9.2

«Si C es una parte de un código, entonces cualquier aserción $\{P\}$ se denomina **precondición** de C si $\{P\}$ *sólo implica el estado inicial*. Cualquier aserción $\{Q\}$ se denomina **postcondición** [de C] si $\{Q\}$ *sólo implica el estado final*. Si C tiene como precondición a $\{P\}$ y como postcondición a $\{Q\}$, se escribe $\{P\} C \{Q\}$. La terna $\{P\} C \{Q\}$ se denomina **terna de Hoare**.»

Conceptos preliminares

Ejemplo (terna de Hoare)

Para la terna de Hoare

$$\{y \neq 0\} \quad x := 1/y \quad \{x = 1/y\},$$

- la precondition $y \neq 0$ afirma que el valor de y es diferente de 0,
- la postcondition $x = 1/y$ afirma que el valor de x es $1/y$,
- y el programa con la precondition y postcondition anteriores es la instrucción $x := 1/y$.

Conceptos preliminares

Ejemplo (precondición vacía)

La terna de Hoare

$$\{\} \ a := b \ \{a = b\},$$

tiene una precondición vacía $\{\}$ la cual se interpreta como «verdadera para todos los estados del programa».

Conceptos preliminares

Pregunta

Las instrucciones `h := a; a := b; b := h` intercambian los valores de las variables `a` y `b`. ¿Cómo escribir una postcondición para estas instrucciones?

Conceptos preliminares

Métodos para establecer la distinción entre los valores de las variables en el estado inicial y en el estado final

(i) Uso de subíndices

El subíndice α se empleará para los valores iniciales de las variables y el subíndice ω se empleará para los valores finales de las variables.

(ii) Uso de variables ocultas

Se utilizan variables que no aparecen en el código para almacenar los valores iniciales de las variables.

Conceptos preliminares

Ejemplos (diferenciando el estado inicial y el estado final)

Ternas de Hoare para las instrucciones que intercambian los valores de dos variables.

1. Empleando los subíndices α y ω

$$\{\} \quad h := a; a := b; b := h \quad \{(a_\omega = b_\alpha) \wedge (b_\omega = a_\alpha)\}.$$

2. Empleando las variables ocultas A y B

$$\{a = A, b = B\} \quad h := a; a := b; b := h \quad \{a = B, b = A\}.$$

Conceptos preliminares

Ejemplo

Una terna de Hoare para una instrucción de asignación.

$$\{\} \quad a := b \quad \{a = b, b = b_{\alpha}\}.$$

Conceptos preliminares

Ejemplo

Una terna de Hoare para una instrucción **if-then-else**.

$\{$

if $x > y$ **then** $m := x$ **else** $m := y$

$\{(x > y \Rightarrow m = x_\alpha) \wedge (\neg(x > y) \Rightarrow m = y_\alpha)\}$.

Conceptos preliminares

Definición 9.3

«Si C es un código con la precondition $\{P\}$ y la postcondición $\{Q\}$, entonces $\{P\} C \{Q\}$ se dice que es **parcialmente correcta** si el estado final de C satisface $\{Q\}$, siempre que el estado inicial satisfaga $\{P\}$. [El programa] C también sería parcialmente correcto si no hay estado final debido a que el programa no termina. Si $\{P\} C \{Q\}$ es parcialmente correcta y C termina, entonces se dice que $\{P\} C \{Q\}$ es **totalmente correcta**.»

Conceptos preliminares

Observación

En el subconjunto de PASCAL que estamos considerando el problema de la terminación solo está asociado a programas que tengan instrucciones **while** .

Conceptos preliminares

Observación

En el subconjunto de PASCAL que estamos considerando el problema de la terminación solo está asociado a programas que tengan instrucciones **while** .

Observación

Observe que la terna de Hoare $\{P\} C \{Q\}$ es **parcialmente correcta** «si cada estado inicial posible que satisfaga P da como resultado un estado final que satisfaga Q .»

Conceptos preliminares

Descripción

La lógica de Hoare (también llamada lógica de Floyd-Hoare) es un sistema formal para construir demostraciones de la corrección parcial de ternas de Hoare $\{P\} C \{Q\}$ compuesta por axiomas y reglas de inferencia.

Reglas generales relativas a las precondiciones y postcondiciones

Definición 9.4

«Si R y S son dos aserciones, entonces se dice que R es **más fuerte** que S si $R \Rightarrow S$. Si R es más fuerte que S entonces se dice que S es **más débil** que R .»

Reglas generales relativas a las precondiciones y postcondiciones

Definición 9.4

«Si R y S son dos aserciones, entonces se dice que R es **más fuerte** que S si $R \Rightarrow S$. Si R es más fuerte que S entonces se dice que S es **más débil** que R .»

Ejemplos

Sean R y S son dos aserciones.

- $R \wedge S$ es más fuerte que R .
- $R \vee S$ es más débil que R .
- $i < 0$ es más fuerte que $i < 1$.
- $i > 1$ es más débil que $i > 0$.

Reglas generales relativas a las precondiciones y postcondiciones

Reforzamiento de precondiciones

$$\frac{P' \Rightarrow P \quad \{P\} C \{Q\}}{\{P'\} C \{Q\}}.$$

Reglas generales relativas a las precondiciones y postcondiciones

Ejemplo 9.2

A partir de la terna de Hoare

$$\{x \geq 0\} \quad y := \text{sqrt}(x) \quad \{y^2 = x\},$$

demostrar que $\{x \geq 1\} \quad y := \text{sqrt}(x) \quad \{y^2 = x\}$.

Reglas generales relativas a las precondiciones y postcondiciones

Ejemplo 9.2

A partir de la terna de Hoare

$$\{x \geq 0\} \text{ y } := \text{sqrt}(x) \{y^2 = x\},$$

demostrar que $\{x \geq 1\} \text{ y } := \text{sqrt}(x) \{y^2 = x\}$.

Demostración

1. $(x \geq 1) \Rightarrow (x \geq 0)$ Aritmética.
2. $\{x \geq 0\} \text{ y } := \text{sqrt}(x) \{y^2 = x\}$ Hipótesis.
3. $\{x \geq 1\} \text{ y } := \text{sqrt}(x) \{y^2 = x\}$ Reforzamiento de precondición, 1,2.



Reglas generales relativas a las precondiciones y postcondiciones

Pregunta

¿Es preferible construir una terna de Hoare con la precondición más débil posible o con la precondición más fuerte posible?

Reglas generales relativas a las precondiciones y postcondiciones

Pregunta

¿Es preferible construir una terna de Hoare con la precondición más débil posible o con la precondición más fuerte posible?

Respuesta

«Por regla general, siempre es ventajoso intentar formular la precondición más débil posible que asegure que se cumple una cierta postcondición dada. Cualquier precondición [más] fuerte será automáticamente satisfecha debido al principio de reforzamiento de precondiciones.[...] El programa que tenga la precondición más débil es el programa más general.» (pág. 465)

Reglas generales relativas a las precondiciones y postcondiciones

Debilitamiento de postcondiciones

$$\frac{\{P\} C \{Q\} \quad Q \Rightarrow Q'}{\{P\} C \{Q'\}}.$$

Reglas generales relativas a las precondiciones y postcondiciones

Ejemplo 9.4

Dada la terna de Hoare

$$\{a > 4\} \quad b := a + 1 \quad \{b > 5, a > 4\},$$

demostrar que $\{a > 4\} \quad b := a + 1 \quad \{b > 5\}$.

Reglas generales relativas a las precondiciones y postcondiciones

Ejemplo 9.4

Dada la terna de Hoare

$$\{a > 4\} \quad b := a + 1 \quad \{b > 5, a > 4\},$$

demostrar que $\{a > 4\} \quad b := a + 1 \quad \{b > 5\}$.

Demostración

1. $\{a > 4\} \quad b := a + 1 \quad \{b > 5, a > 4\}$

Hipótesis.

2. $(b > 5, a > 4) \Rightarrow [(b > 5) \wedge (a > 4)]$

Ley de combinación, tabla 1.25.

3. $[(b > 5) \wedge (a > 4)] \Rightarrow (b > 5)$

Ley de simplificación, tabla 1.25.

4. $\{a > 4\} \quad b := a + 1 \quad \{b > 5\}$

Debilitamiento de postcondición, 1,3.



Reglas generales relativas a las precondiciones y postcondiciones

Pregunta

¿Es preferible construir una terna de Hoare con la postcondición más débil posible o con la postcondición más fuerte posible?

Reglas generales relativas a las precondiciones y postcondiciones

Pregunta

¿Es preferible construir una terna de Hoare con la postcondición más débil posible o con la postcondición más fuerte posible?

Respuesta

«Si es posible, se debe intentar encontrar postcondiciones tan específicas o tan fuertes como sea posible, especialmente cuando una parte de un código se utilice en varios lugares diferentes. Si en una aplicación en particular una postcondición más débil resulta aceptable, siempre se puede utilizar el debilitamiento de la postcondición.» (pág. 467)

Reglas generales relativas a las precondiciones y postcondiciones

Regla de conjunción

$$\frac{\begin{array}{c} \{P_1\} C \{Q_1\} \\ \{P_2\} C \{Q_2\} \end{array}}{\{P_1 \wedge P_2\} C \{Q_1 \wedge Q_2\}}.$$

Reglas generales relativas a las precondiciones y postcondiciones

Observación

La precondición vacía representa a $\mathbf{V}$ y puesto que $P \equiv P \wedge \mathbf{V}$ (ley de identidad, tabla 1.16), la regla de conjunción se puede aplicar de la siguiente manera:

$$\frac{\begin{array}{c} \{\} C \{Q_1\} \\ \{P\} C \{Q_2\} \end{array}}{\{P\} C \{Q_1 \wedge Q_2\}}.$$

Reglas generales relativas a las precondiciones y postcondiciones

Ejemplo 9.5

Aplicar las ternas de Hoare

$$\begin{aligned} & \{\} \quad i := i + 1 \quad \{i_\omega = i_\alpha + 1\}, \\ & \{i_\alpha > 0\} \quad i := i + 1 \quad \{i_\alpha > 0\}, \end{aligned}$$

para demostrar que

$$\{i > 0\} \quad i := i + 1 \quad \{i > 1\}.$$

(continua en la próxima diapositiva)

Reglas generales relativas a las precondiciones y postcondiciones

Demostración

1. $\{\} \quad i := i + 1 \quad \{i_\omega = i_\alpha + 1\}$
2. $\{i_\alpha > 0\} \quad i := i + 1 \quad \{i_\alpha > 0\}$
3. $\{i_\alpha > 0\} \quad i := i + 1$
 $\{(i_\omega = i_\alpha + 1) \wedge (i_\alpha > 0)\}$
4. $\quad (i_\omega = i_\alpha + 1) \wedge (i_\alpha > 0)$
5. $\quad (i_\alpha = i_\omega - 1) \wedge (i_\alpha > 0)$
6. $\quad (i_\alpha = i_\omega - 1) \wedge (i_\omega - 1 > 0)$
7. $\quad i_\omega - 1 > 0$
8. $\quad i_\omega > 1$
9. $[(i_\omega = i_\alpha + 1) \wedge (i_\alpha > 0)] \Rightarrow (i_\omega > 1)$
10. $\{i_\alpha > 0\} \quad i := i + 1 \quad \{i_\omega > 1\}$
11. $\{i > 0\} \quad i := i + 1 \quad \{i > 1\}$

Hipótesis.

Hipótesis.

Regla de conjunción, 1,2.

Supuesto.

Aritmética.

Sustitución.

Ley de simplificación, 6, tabla 1.25.

Aritmética.

Demostración condicional, 4–8.

Debilitamiento de postcond., 3,9.

Los subíndices α y ω se refieren al estado actual.

Reglas generales relativas a las precondiciones y postcondiciones

Observación

Por la ley de combinación de la tabla 1.16

$$A, B \models A \wedge B,$$

la regla de conjunción se puede por ejemplo aplicar de la siguiente manera:

$$\frac{\begin{array}{c} \{P_1, P_2\} C \{Q_1, Q_2\} \\ \{R\} C \{S\} \end{array}}{\{P_1, P_2, R\} C \{Q_1, Q_2, S\}}.$$

Reglas generales relativas a las precondiciones y postcondiciones

Ejemplo 9.6

Dada la terna de Hoare

$$\{ \} \quad h := a; a := b; b := h \quad \{ a = b_\alpha, b = a_\alpha \},$$

demostrar que

$$\{ a > b \} \quad h := a; a := b; b := h \quad \{ b > a \}.$$

(continua en la próxima diapositiva)

Reglas generales relativas a las precondiciones y postcondiciones

Demostración

1. $\{ \} \text{ h := a; a := b; b := h }$

$\{a = b_\alpha, b = a_\alpha\}$

Hipótesis.

2. $\{a_\alpha > b_\alpha\} \text{ h := a; a := b; b := h }$

$\{a_\alpha > b_\alpha\}$

Los valores iniciales no cambian.

3. $\{a_\alpha > b_\alpha\} \text{ h := a; a := b; b := h }$

$\{a = b_\alpha, b = a_\alpha, a_\alpha > b_\alpha\}$

Regla de conjunción, 1,2.

4. $a = b_\alpha, b = a_\alpha, (a_\alpha > b_\alpha) \Rightarrow (b > a)$

Aritmética.

5. $\{a_\alpha > b_\alpha\} \text{ h := a; a := b; b := h }$

$\{b > a\}$

Debilitamiento de postcond., 3,4.

6. $\{a > b\} \text{ h := a; a := b; b := h }$

$\{b > a\}$

El índice α se refiere al estado inicial.



Reglas generales relativas a las precondiciones y postcondiciones

Regla de disyunción

$$\frac{\begin{array}{c} \{P_1\} C \{Q_1\} \\ \{P_2\} C \{Q_2\} \end{array}}{\{P_1 \vee P_2\} C \{Q_1 \vee Q_2\}}.$$

Reglas generales relativas a las precondiciones y postcondiciones

Observación

La precondición vacía representa a $\mathbf{V}$ y puesto que $P \vee \mathbf{V} \equiv \mathbf{V}$ Ley de dominación, tabla 1.16), la regla de disyunción se puede aplicar de la siguiente manera:

$$\frac{\begin{array}{c} \{\} C \{Q_1\} \\ \{P\} C \{Q_2\} \end{array}}{\{\} C \{Q_1 \vee Q_2\}}.$$

Verificación de código sin bucles

Observación

«Para demostrar la corrección de las partes de un programa que contienen varias sentencias, se utiliza como punto de partida la postcondición de todo el código. Esto es muy normal. En primer lugar, uno debe preguntarse qué resultados hay que obtener y qué condiciones deben satisfacer estos resultados y esto viene expresado por la postcondición. Luego la precondition se deduce de la postcondición, Por esta razón, el código se verifica en el orden contrario a como se ejecuta, La postcondición de la última sentencia es idéntica a la postcondición de todo el código. Después se deduce la precondition de última la sentencia, que se convierte en la postcondición de la sentencia previa. Esto permite encontrar las precondiciones desde la segunda sentencia hasta la última, que se convierten a su vez en las postcondiciones de la tercera a la última. Se continua de esta forma hasta que se encuentra la precondition de la primera sentencia, que se convierte en la precondition de todo el código.» (pág. 470).

Verificación de código sin bucles

Regla de asignación

«Sea E una expresión y V una variable sin subíndice. Además, si C es una sentencia de la forma $V := E$ con la postcondición $\{Q\}$, entonces la precondition de C puede hallarse sustituyendo todos los casos de V en Q por E . Si Q_E^V es la expresión que se obtiene mediante esto, se sigue lo siguiente:»

$$\{Q_E^V\} V := E \{Q\}.$$

Verificación de código sin bucles

Ejemplo 9.7

Considérese la sentencia $j := i + 1$. Supóngase que la postcondición de esta sentencia es $j > 0$. Hallar la precondition.

Verificación de código sin bucles

Ejemplo 9.7

Considérese la sentencia $j := i + 1$. Supóngase que la postcondición de esta sentencia es $j > 0$. Hallar la precondition.

Solución

$\{i + 1 > 0\} \quad j := i + 1 \quad \{j > 0\}$ Regla de asignación.

Verificación de código sin bucles

Ejemplo 9.10

Hallar la precondition de la sentencia $x := 1/x$, sabiendo que la postcondición es $x \geq 0$.

Verificación de código sin bucles

Ejemplo 9.10

Hallar la precondition de la sentencia $x := 1/x$, sabiendo que la postcondición es $x \geq 0$.

Solución

1. $\{1/x \geq 0\} \quad x := 1/x \quad \{x \geq 0\}$
2. $\{x \neq 0\} \quad x := 1/x \quad \{x \geq 0\}$
3. $\{(1/x \geq 0) \wedge (x \neq 0)\} \quad x := 1/x \quad \{x \geq 0\}$
4. $(x > 0) \Rightarrow (1/x > 0)$
5. $(1/x > 0) \Rightarrow (1/x \geq 0)$
6. $(x > 0) \Rightarrow (1/x \geq 0)$
7. $(x > 0) \Rightarrow (x \neq 0)$
8. $(x > 0) \Rightarrow [(1/x \geq 0) \wedge (x \neq 0)]$
9. $\{x > 0\} \quad x := 1/x \quad \{x \geq 0\}$

Regla de asignación.

Precond. para ejecutar el programa.

Regla de conjunción, 1,2.

Aritmética.

Ley de adición, tabla 1.25 y def. $\geq$.

Silogismo hip., 4,5, tabla 1.25.

Aritmética

Lógica, si $A \models B$ y $A \models C$ entonces $A \models B \wedge C$.

Reforzamiento de precond., 8,3.

Verificación de código sin bucles

Regla de concatenación

$$\frac{\{P\} C_1 \{R\} \quad \{R\} C_2 \{Q\}}{\{P\} C_1; C_2 \{Q\} .}$$

Verificación de código sin bucles

Ejemplo 9.11

Demostrar que el siguiente código es correcto.

$$\{\} \quad c := a + b; \quad c := c/2 \quad \left\{ c = \frac{a+b}{2} \right\}.$$

Verificación de código sin bucles

Ejemplo 9.11

Demostrar que el siguiente código es correcto.

$$\{\} \quad c := a + b; \quad c := c/2 \quad \left\{ c = \frac{a+b}{2} \right\}.$$

Demostración

1. $\{\} \quad c := a + b \quad \{c = a + b\}$

Regla de asignación.

2. $\{c = a + b\} \quad c := c/2 \quad \left\{ c = \frac{a+b}{2} \right\}$

Regla de asignación.

3. $\{\} \quad c := a + b; \quad c := c/2 \quad \left\{ c = \frac{a+b}{2} \right\}$

Regla de concatenación, 1,2.



Verificación de código sin bucles

Ejemplo 9.12

El siguiente código no tiene precondition alguna y su postcondición es $s = 1 + r + r^2$. Demostrar que el código es correcto.

```
s := 1;  
s := s + r;  
s := s + r * r
```

(continua en la próxima diapositiva)

Verificación de código sin bucles

Solución

Precondición	Sentencia	Postcondición	Regla
		$\{\}$	
$\{1 = 1\}$	$s := 1$	$\{s = 1\}$	Regla de asignación
$\{s + r = 1 + r\}$	$s := s + r$	$\{s = 1 + r\}$	Regla de asignación
$\{s + r^2 = 1 + r + r^2\}$	$s := s + r * r$	$\{s = 1 + r + r^2\}$	Regla de asignación

(continua en la próxima diapositiva)

Verificación de código sin bucles

Solución

Precondición	Sentencia	Postcondición	Regla
		$\{\}$	
$\{1 = 1\}$	$s := 1$	$\{s = 1\}$	Regla de asignación
$\{s + r = 1 + r\}$	$s := s + r$	$\{s = 1 + r\}$	Regla de asignación
$\{s + r^2 = 1 + r + r^2\}$	$s := s + r * r$	$\{s = 1 + r + r^2\}^a$	Regla de asignación

(continua en la próxima diapositiva)

^aVéase las correcciones al texto guía en la página web del curso.

Verificación de código sin bucles

Solución (continuación)

La terna final de Hoare es:

$\{\}$

$s := 1;$

$s := s + r;$

$s := s + r * r$

$\{s = 1 + r + r^2\}$

Verificación de código sin bucles

Definición

Un **invariante** es una aserción que es a la vez precondition y postcondición de una parte de un código.

Verificación de código sin bucles

Ejemplo 9.14

Demostrar que $\{r = 2^i\}$ es un invariante de `i := i + 1; r := r * 2.`

Verificación de código sin bucles

Ejemplo 9.14

Demostrar que $\{r = 2^i\}$ es un invariante de $i := i + 1; r := r * 2$.

Solución

Precondición	Sentencia	Postcondición	Regla
$\{r = 2^i\}$	$i := i + 1$	$\{r = 2^i\}$ $\{r = 2^{i-1}\}$	Regla de asignación
$\{2r = 2^i\}$	$r = r * 2$	$\{r = 2^i\}$	Regla de asignación

Verificación de código sin bucles

Ejemplo 9.14

Demostrar que $\{r = 2^i\}$ es un invariante de $i := i + 1; r := r * 2$.

Solución

Precondición	Sentencia	Postcondición	Regla
$\{r = 2^i\}$	$i := i + 1$	$\{r = 2^i\}$ $\{r = 2^{i-1}\}$	Regla de asignación
$\{2r = 2^i\}$	$r = r * 2$	$\{r = 2^i\}$	Regla de asignación

Es decir, la terna final de Hoare es

$$\{r = 2^i\} \quad i := i + 1; r = r * 2 \quad \{r = 2^i\}.$$

Verificación de código sin bucles

Ejemplo 9.14

Demostrar que $\{r = 2^i\}$ es un invariante de $i := i + 1; r := r * 2$.

Solución

Precondición	Sentencia	Postcondición	Regla
$\{r = 2^i\}$	$i := i + 1$	$\{r = 2^i\}$ $\{r = 2^{i-1}\}$	Regla de asignación
$\{2r = 2^i\}^a$	$r = r * 2$	$\{r = 2^i\}$	Regla de asignación

Es decir, la terna final de Hoare es

$$\{r = 2^i\} \quad i := i + 1; r = r * 2 \quad \{r = 2^i\}.$$

^aVéase las correcciones al texto guía en la página web del curso.

Verificación de código sin bucles

Regla para condiciones con cláusula else

$$\frac{\begin{array}{c} \{P \wedge B\} C_1 \{Q\} \\ \{P \wedge \neg B\} C_2 \{Q\} \end{array}}{\{P\} \text{ if } B \text{ then } C_1 \text{ else } C_2 \{Q\} .}$$

Verificación de código sin bucles

Ejemplo 9.15

Demostrar que

$$\{\} \text{ if } a > b \text{ then } m := a \text{ else } m := b \{ (m \geq a) \wedge (m \geq b) \}.$$

Verificación de código sin bucles

Ejemplo 9.15

Demostrar que

$$\{\} \text{ if } a > b \text{ then } m := a \text{ else } m := b \{ (m \geq a) \wedge (m \geq b) \}.$$

Demostración

En el tablero.

Verificación de código sin bucles

Regla para condiciones

$$\frac{\begin{array}{l} \{P \wedge B\} C \{Q\} \\ (P \wedge \neg B) \Rightarrow Q \end{array}}{\{P\} \text{ **if B then C** } \{Q\} .}$$

Verificación de código sin bucles

Ejemplo 9.16

Demostrar que, para cualquier precondition posible, el siguiente fragmento de programa tiene como postcondición $\{\text{max} \geq a\}$.

```
if max < a then max := a
```

Verificación de código sin bucles

Ejemplo 9.16

Demostrar que, para cualquier precondition posible, el siguiente fragmento de programa tiene como postcondición $\{\max \geq a\}$.

```
if max < a then max := a
```

Demostración

En el tablero.

Verificación de código sin bucles

Ejemplo

A partir de las reglas de la lógica de Hoare conocidas hasta el momento, demostrar la siguiente regla:

$$\frac{\begin{array}{l} \{B\} C_1 \{B\} \\ \{\neg B\} C_2 \{\neg B\} \\ \{P\} C_1 \{Q_1\} \\ \{P\} C_2 \{Q_2\} \end{array}}{\{P\} \text{ if } B \text{ then } C_1 \text{ else } C_2 \{(B \wedge Q_1) \vee (\neg B \wedge Q_2)\} .}$$

(continua en la próxima diapositiva)

Verificación de código sin bucles

Demostración

1. $\{B\} C_1 \{B\}$
2. $\{\neg B\} C_2 \{\neg B\}$
3. $\{P\} C_1 \{Q_1\}$
4. $\{P\} C_2 \{Q_2\}$
5. $\{B \wedge P\} C_1 \{B \wedge Q_1\}$
6. $(B \wedge Q_1) \Rightarrow [(B \wedge Q_1) \vee (\neg B \wedge Q_2)]$
7. $\{B \wedge P\} C_1 \{(B \wedge Q_1) \vee (\neg B \wedge Q_2)\}$
8. $\{\neg B \wedge P\} C_2 \{\neg B \wedge Q_2\}$
9. $(\neg B \wedge Q_2) \Rightarrow [(B \wedge Q_1) \vee (\neg B \wedge Q_2)]$
10. $\{\neg B \wedge P\} C_2 \{(B \wedge Q_1) \vee (\neg B \wedge Q_2)\}$
11. $\{P\} \text{ if } B \text{ then } C_1 \text{ else } C_2$
 $\{(B \wedge Q_1) \vee (\neg B \wedge Q_2)\}$

Hipótesis.

Hipótesis.

Hipótesis.

Hipótesis.

Regla de conjunción, 1,3.

Ley de adición, tabla 1.25.

Debilitamiento de postcond, 5,6.

Regla de conjunción, 2,4.

Ley de adición, tabla 1.25-

Debilitamiento de postcond., 8,9.

Regla para cond. con cláusula else, 7,10.

Bucles y arreglos

Introducción

```
1 suma := 0;  
2 j := 0;  
3 while j <> n do  
4   begin  
5     sum := sum + a;  
6     j := j + 1;  
7   end
```

Para el código arriba tenemos que:

Condición de entrada: $j \neq n$

Condición de salida: $\neg(j \neq n)$, es decir, $j = n$

Postcondición: $sum = n * a$

Invariante: $sum = j * a$

Variante: $n - j$

Bucles y arreglos

Regla while preliminar

$$\frac{\{I\} C \{I\}}{\{I\} \text{ while } D \text{ do } C \{ \neg D \wedge I \} .}$$

Verificación de código sin bucles

Ejemplo 9.18

Utilizar la regla **while** para demostrar que el siguiente código es correcto respecto a la precondition $\{\}$ y la postcondition $\{sum = n * a\}$.

```
1 suma := 0;  
2 j := 0;  
3 while j <> n do  
4   begin  
5     sum := sum + a;  
6     j := j + 1;  
7   end
```

Verificación de código sin bucles

Ejemplo 9.18

Utilizar la regla **while** para demostrar que el siguiente código es correcto respecto a la precondition $\{\}$ y la postcondition $\{sum = n * a\}$.

```
1  suma := 0;  
2  j := 0;  
3  while j <> n do  
4      begin  
5          sum := sum + a;  
6          j := j + 1;  
7      end
```

Demostración

En el tablero.

Verificación de código sin bucles

Ejemplo 9.19

En el siguiente código, hallar el invariante del bucle y la postcondición. El código se supone que halla la suma de cuadrados de 1 a n .

```
1  i := 1;  
2  sum := 0;  
3  while not (i = n + 1) do  
4    begin  
5      sum := sum + i * i;  
6      i := i + 1;  
7    end
```

Verificación de código sin bucles

Ejemplo 9.19

En el siguiente código, hallar el invariante del bucle y la postcondición. El código se supone que halla la suma de cuadrados de 1 a n .

```
1  i := 1;  
2  sum := 0;  
3  while not (i = n + 1) do  
4    begin  
5      sum := sum + i * i;  
6      i := i + 1;  
7    end
```

Demostración

En el tablero.

Resumen: Reglas de la lógica de Hoare

- Reforzamiento de precondiciones

$$\frac{P' \Rightarrow P \quad \{P\} C \{Q\}}{\{P'\} C \{Q\}}$$

- Regla de conjunción

$$\frac{\{P_1\} C \{Q_1\} \quad \{P_2\} C \{Q_2\}}{\{P_1 \wedge P_2\} C \{Q_1 \wedge Q_2\}}$$

- Debilitamiento de postcondiciones

$$\frac{\{P\} C \{Q\} \quad Q \Rightarrow Q'}{\{P\} C \{Q'\}}$$

- Regla de disyunción

$$\frac{\{P_1\} C \{Q_1\} \quad \{P_2\} C \{Q_2\}}{\{P_1 \vee P_2\} C \{Q_1 \vee Q_2\}}$$

(continua en la próxima diapositiva)

Resumen: Reglas de la lógica de Hoare

- Regla de asignación

$$\{Q_E^V\} V := E \{Q\}$$

- Regla de concatenación

$$\frac{\{P\} C_1 \{R\} \quad \{R\} C_2 \{Q\}}{\{P\} C_1; C_2 \{Q\}}$$

- Regla para condiciones con cláusula else
- Regla para condiciones

$$\frac{\{P \wedge B\} C_1 \{Q\} \quad \{P \wedge \neg B\} C_2 \{Q\}}{\{P\} \text{ if } B \text{ then } C_1 \text{ else } C_2 \{Q\} .}$$

$$\frac{\{P \wedge B\} C \{Q\} \quad (P \wedge \neg B) \Rightarrow Q}{\{P\} \text{ if } B \text{ then } C \{Q\} .}$$

(continua en la próxima diapositiva)

Resumen: Reglas de la lógica de Hoare

- Regla while preliminar

$$\frac{\{I\} C \{I\}}{\{I\} \text{ while } D \text{ do } C \{ \neg D \wedge I \} .}$$

- Regla while general

$$\frac{\{D \wedge I\} C \{I\}}{\{I\} \text{ while } D \text{ do } C \{ \neg D \wedge I \} .}$$

Sumatorias

Definición

Sea $a_1, a_2, \dots, a_n$ una secuencia finita de números, donde n es un entero positivo. Recordemos la definición recursiva de la **notación de sumatoria**:

$$\sum_{k=1}^1 a_k := a_1,$$

$$\begin{aligned}\sum_{k=1}^n a_k &:= \left(\sum_{k=1}^{n-1} a_k \right) + a_n \\ &= a_1 + a_2 + \cdots + a_{n-1} + a_n.\end{aligned}$$

Sumatorias

Propiedades

$$\sum_{k=1}^n (a_k + b_k) = \sum_{k=1}^n a_k + \sum_{k=1}^n b_k$$

(aditividad),

$$\sum_{k=1}^n ca_k = c \sum_{k=1}^n a_k$$

(homogeneidad),

$$\sum_{k=1}^n (\alpha a_k + \beta b_k) = \alpha \sum_{k=1}^n a_k + \beta \sum_{k=1}^n b_k$$

(linealidad).

Sumatorias

Propiedades

$$\sum_{k=1}^n f(n) = nf(n),$$

$$\sum_{k=1}^0 f(k) = 0,$$

$$\sum_{k=1}^n a_k = \sum_{k=1}^i a_k + \sum_{k=i+1}^n a_k, \text{ con } i < n.$$

Sumatorias

Propiedades

$$\sum_{k=1}^n k = \frac{n(n+1)}{2},$$

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6},$$

$$\sum_{k=1}^n k^3 = \left(\frac{n(n+1)}{2} \right)^2.$$

Referencias



Winfried Karl Grassman y Jean-Paul Tremblay [1996] (2003). Matemáticas Discretas y Lógica. Una Perspectiva desde la Ciencia de la Computación. Trad. por Rafael García-Bermejo, María Luisa Díez Platas y Vivian de los Ángeles Fernández Vásquez. Última reimpresión. Prentice Hall (vid. pág. 2).